LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 2

STACK DAN QUEUE

Oleh:

Nuzulia Sekti Mahanania NIM. 2510817220006

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 2

Laporan Praktikum Algoritma & Struktur Data Modul 2: Stack dan Queue ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Prakitkum ini dikerjakan oleh:

Nama Praktikan : Nuzulia Sekti Mahanania
NIM : 2510817220006

Menyetujui, Asisten Praktikum	Mengetahui, Dosen Penanggung Jawab Praktikum
Muhammad Fauzan Ahsani NIM. 2310817310009	Ir. Muhamamd Alkaff, S,Kom, M.Kom, Ph.D NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN i
DAFTAR ISI ii
DAFTAR GAMBAR iii
DAFTAR TABEL iv
SOAL 1 : Implementasi Stack 5
A. Source Code 5
B. Pembahasan 6
SOAL 2 : Si Palui dan Mesin Hitung Aneh 8
A. Source Code 9
B. Output Program 10
C. Pembahasan 10
SOAL 3 : Implementasi Queue 12
A. Source Code 12
B. Pembahasan 13
SOAL 4 : Si Palui dan Deretan Angka 15
A. Source Code 16
B. Output Program 17
C. Pembahasan 17
TAUTAN GITHUB 19

DAFTAR GAMBAR

Gambar 1. Screenshot Output Soal 2 10
Gambar 2. Screenshot Output Soal 4 17

DAFTAR TABEL

Tabel 1. Source Code stack.cpp 5
Tabel 2. Source Code prob1.cpp 9
Tabel 3. Source Code queue.cpp 12
Tabel 4. Source Code prob2.cpp 16

SOAL 1 : Implementasi Stack

Pada file `stack.h` yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file `stack.cpp`. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan `throw`.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Tabel 1. Source Code stack.cpp

```
#include "stack.h"
#include <stdexcept>

void init(Stack* s) {
    s->top = nullptr;
}

bool isEmpty(const Stack* s) {
    return s->top == nullptr;
}

bool isFull(const Stack* s) {
    if (isEmpty(s)) return false;
    return s->top == s->data + MAX - 1;
}

void push(Stack* s, int value) {
    if (isFull(s)) {
        throw std::overflow_error("Stack is full");
    }

    if (isEmpty(s)) {
        s->top = s->data;
    } else {
        s->top++;
    }
    *(s->top) = value;
}

void pop(Stack* s) {
```

```
    if (isEmpty(s)) {
        throw std::underflow_error("Stack is empty");
    }

    if (s->top == s->data) {
        s->top = nullptr;
    } else {
        s->top--;
    }
}

int peek(const Stack* s) {
    if (isEmpty(s)) {
        throw std::underflow_error("Stack is empty");
    }
    return *(s->top);
}
```

B. Pembahasan

1. Analisis Implementasi Stack

Program ini mengimplementasikan struktur data *Stack* menggunakan *array* statis dengan kendali pointer `top` sebagai penanda posisi elemen teratas. Penggunaan pointer memberikan efisiensi tinggi karena akses data dilakukan langsung melalui alamat memori. Inisialisasi awal dilakukan dengan mengatur `top` ke `nullptr`, yang secara logis menandakan bahwa tumpukan belum menunjuk ke elemen manapun dalam array.

2. Mekanisme Kerja dan Justifikasi

Kebenaran algoritma ini didasarkan pada prinsip *Last-In-First-Out* (LIFO). Pada operasi `push`, pointer `top` akan bergerak maju ke alamat memori berikutnya sebelum data baru disimpan, kecuali jika tumpukan sudah mencapai batas alamat elemen terakhir (`isFull`). Sebaliknya, pada operasi `pop`, pointer `top` akan dimundurkan satu posisi, yang secara otomatis membuat data lama tidak dapat diakses lagi dan akan tertimpa saat penambahan data berikutnya dilakukan. Untuk menjaga keamanan program, setiap kondisi tidak valid seperti mencoba menambah data pada tumpukan penuh atau mengambil data dari tumpukan kosong ditangani menggunakan mekanisme *exception handling* (`throw`).

3. Kompleksitas dan Notasi O(1)

Seluruh fungsi dalam program ini memiliki kompleksitas Waktu O(1) dan Ruang O(1). Notasi O(1) atau Waktu Konstan berarti kecepatan eksekusi program akan selalu sama dan stabil, tidak peduli seberapa banyak data yang tersimpan di dalam tumpukan. Hal ini terjadi

karena program tidak melakukan proses pencarian atau perulangan, melainkan langsung mengakses satu titik memori tertentu melalui pointer.

SOAL 2 : Si Palui dan Mesin Hitung Aneh

Time Limit: 1.0 s
Memory Limit: 64 MB

Si Palui menemukan sebuah mesin hitung tua di gudang. Mesin tersebut memiliki cara kerja yang tidak biasa. Alih-alih menggunakan format perhitungan seperti yang biasa kita kenal, mesin ini membaca input berupa deretan simbol yang dipisahkan oleh spasi.

Setiap simbol bisa berupa:

- sebuah bilangan bulat, atau
- sebuah operator (+, -, *, /)

Cara kerja mesin adalah sebagai berikut:

- Mesin membaca simbol dari kiri ke kanan.
- Jika simbol adalah bilangan, maka bilangan tersebut disimpan ke dalam memori mesin.
- Jika simbol adalah operator, maka mesin akan mengambil dua nilai yang paling baru dimasukkan ke dalam memori yang disimpan, melakukan operasi sesuai operator tersebut, lalu menyimpan kembali hasilnya ke dalam memori.

Setelah seluruh simbol diproses, akan tersisa satu nilai di dalam memori. Nilai inilah yang ditampilkan sebagai hasil akhir.

Sebagai asisten Si Palui, tugas Anda adalah membantu menghitung hasil dari mesin tersebut.

Batasan

- $1 \leq n \leq 100$
- $-1000 \leq Ai \leq 1000$

Keterangan:

- n adalah jumlah simbol dalam ekspresi
- Ai adalah nilai bilangan pada input
- Simbol yang valid hanya:
  - bilangan bulat
  - operator: +, -, *, /

Format Input

n

A1 A2 ... An

Keterangan:

- Baris pertama n menyatakan jumlah atau panjang simbol
- Baris kedua berisi simbol A sebanyak n buah

Format Output

Sebuah bilangan hasil dari kalkulator Si Palui.

Contoh Kasus

Input	Output
3 2 3 +	5
9 5 1 2 + 4 * + 3 -	14
5 10 2 / 3 *	15

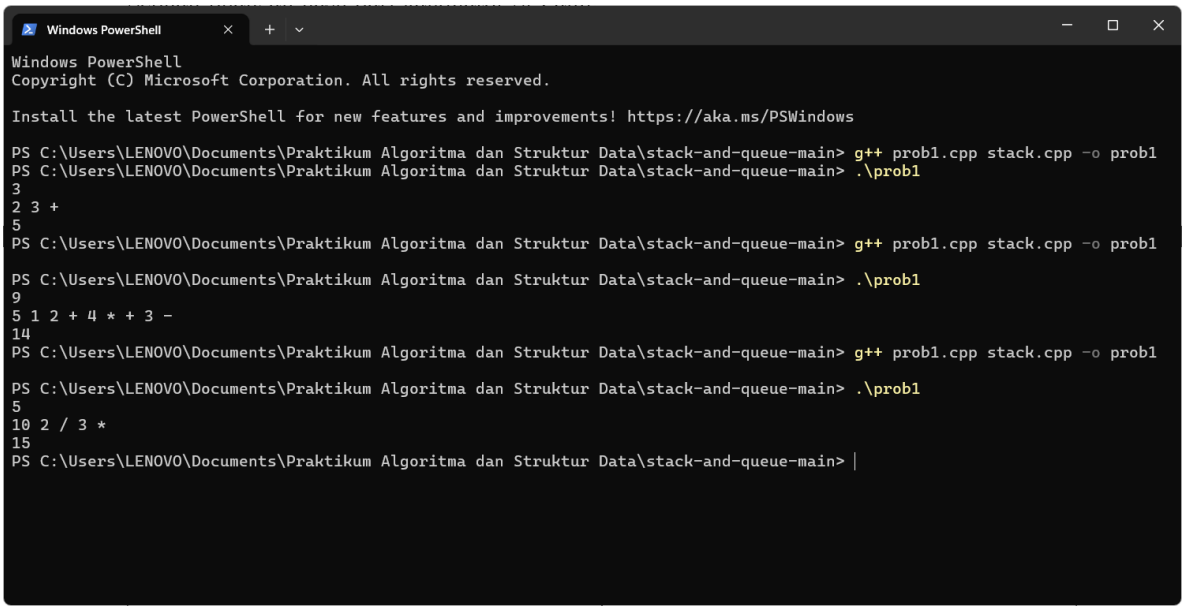
A. Source Code

Tabel 2. Source Code prob1.cpp

```
#include <iostream>
#include <string>
#include <cctype>
#include "stack.h"

using namespace std;

int main() {
    int n;
    if (!(cin >> n)) return 0;

    Stack s;
    init(&s);

    for (int i = 0; i < n; i++) {
        string token;
        cin >> token;

        if (isdigit(token[0]) || (token.length() > 1 && token[0] == '-')) {
            push(&s, stoi(token));
        } else {
            int val2 = peek(&s); pop(&s);
```

```
23                int val1 = peek(&s); pop(&s);
24                if (token == "+") push(&s, val1 + val2);
25                else if (token == "-") push(&s, val1 - val2);
26                else if (token == "*") push(&s, val1 * val2);
27                else if (token == "/") push(&s, val1 / val2);
28            }
29        }
30
31        cout << peek(&s) << endl;
32        return 0;
33    }
```

B. Output Program



Gambar 1. Screenshot Output Soal 2

C. Pembahasan

1. Penjelasan Baris Kode

Baris [1-4]: Bagian ini digunakan untuk menyertakan pustaka (*library*) yang dibutuhkan. `iostream` untuk input/output, `string` untuk mengelola teks, `cctype` untuk fungsi pengecekan karakter, dan `stack.h` untuk memanggil fungsi-fungsi tumpukan yang telah dibuat sebelumnya.

Baris [6-8]: Perintah `using namespace std` digunakan agar kita tidak perlu menuliskan awalan `std::` pada fungsi umum seperti `cin` atau `cout`. Baris 8 merupakan fungsi utama (`main`) di mana eksekusi program dimulai.

Baris [9-10]: Program mendeklarasikan variabel `n` untuk menyimpan jumlah simbol yang akan dimasukkan. Perintah `if (!(cin >> n))` berfungsi untuk membaca input tersebut dan memastikan program langsung berhenti jika input tidak valid.

Baris [12-13]: Baris ini mendeklarasikan variabel `s` dengan tipe data `Stack` dan memanggil fungsi `init(&s)` untuk menyiapkan tumpukan dalam keadaan kosong sebelum digunakan.

Baris [15-17]: Program memulai perulangan (*loop*) sebanyak `n` kali untuk membaca setiap simbol (angka atau operator) satu per satu. Setiap simbol dibaca dan disimpan ke dalam variabel string bernama `token`.

Baris [19-20]: Bagian ini melakukan pengecekan apakah `token` yang dibaca adalah sebuah angka. Kondisi `isdigit` mengecek angka positif, sedangkan pengecekan tanda `-` digunakan untuk menangani angka negatif. Jika benar angka, string tersebut diubah menjadi integer menggunakan `stoi` dan dimasukkan ke dalam tumpukan dengan fungsi `push`.

Baris [21-23]: Jika `token` bukan angka, maka dianggap sebagai operator. Program mengambil dua angka teratas dari tumpukan: angka terakhir yang masuk disimpan ke `val2`, dan angka sebelumnya disimpan ke `val1`. Kedua angka tersebut kemudian dikeluarkan dari tumpukan menggunakan fungsi `pop`.

Baris [24-27]: Program mengecek jenis operator yang ada pada `token`. Jika operator adalah `+`, `-`, `*`, atau `/`, program akan melakukan perhitungan antara `val1` dan `val2` sesuai operator tersebut, lalu memasukkan hasilnya kembali ke dalam tumpukan.

Baris [31-32]: Setelah semua simbol selesai diproses dalam perulangan, hasil akhir perhitungan akan berada di posisi paling atas tumpukan. Program mencetak hasil tersebut menggunakan `peek(&s)` dan mengakhiri jalannya program dengan `return 0`

2. Analisis Kompleksitas

Kompleksitas Waktu O(n) :
Program melakukan perulangan sebanyak n kali sesuai jumlah simbol. Di dalam setiap perulangan, operasi tumpukan (`push, pop, peek`) berjalan dalam waktu O(1) (Waktu Konstan), sehingga total waktunya bergantung secara linear pada jumlah input.

Kompleksitas Ruang O(1):
Karena penggunaan memori ini tidak bertambah atau berkurang mengikuti besar kecilnya input n dari pengguna, maka kompleksitas ruangnya dikategorikan sebagai Waktu Konstan atau O(1).

SOAL 3 : Implementasi Queue

Pada file `queue.h` yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file `queue.cpp`. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan throw.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Tabel 3. Source Code queue.cpp

```
#include "queue.h"
#include <stdexcept>

void init(Queue* q) {
    q->front = nullptr;
    q->rear = nullptr;
}

bool isEmpty(const Queue* q) {
    return q->front == nullptr;
}

bool isFull(const Queue* q) {
    if (isEmpty(q)) return false;

    const int* next_rear = q->data + ((q->rear - q->data + 1) % MAX);

    return next_rear == q->front;
}

void enqueue(Queue* q, int value) {
    if (isFull(q)) throw std::overflow_error("Queue is full");

    if (isEmpty(q)) {
        q->front = q->data;
        q->rear = q->data;
    } else {
```

```
        q->rear = q->data + ((q->rear - q->data + 1) % MAX);
    }
    *(q->rear) = value;
}

void dequeue(Queue* q) {
    if (isEmpty(q)) throw std::underflow_error("Queue is empty");

    if (q->front == q->rear) {
        q->front = nullptr;
        q->rear = nullptr;
    } else {
        q->front = q->data + ((q->front - q->data + 1) % MAX);
    }
}

int front(const Queue* q) {
    if (isEmpty(q)) throw std::underflow_error("Queue is empty");
    return *(q->front);
}

int back(const Queue* q) {
    if (isEmpty(q)) throw std::underflow_error("Queue is empty");
    return *(q->rear);
}
```

B. Pembahasan

1. Analisis Implementasi Stack

Implementasi ini menggunakan struktur data *Circular Queue* (antrean melingkar) yang dibangun di atas *array* statis dengan memanfaatkan dua pointer utama, yaitu `front` dan `rear`. Berbeda dengan antrean linier biasa, antrean melingkar memungkinkan penggunaan kembali ruang kosong di bagian depan *array* setelah operasi penghapusan dilakukan, sehingga lebih efisien dalam penggunaan memori.

2. Mekanisme Kerja dan Justifikasi

Inisialisasi dan Pemeriksaan Kondisi

Proses dimulai dengan fungsi `init` yang mengatur pointer `front` dan `rear` menjadi `nullptr`. Hal ini memberikan dasar bagi fungsi `isEmpty` untuk memverifikasi kekosongan antrean cukup dengan memeriksa apakah `front` bernilai `nullptr`. Pada fungsi `isFull`, algoritma menghitung posisi "berikutnya" dari `rear` menggunakan operasi modulo (`% MAX`) terhadap alamat dasar array. Jika alamat hasil perhitungan tersebut sama dengan alamat yang ditunjuk oleh `front`, maka antrean dinyatakan penuh. Justifikasi kebenaran ini terletak pada logika yang di mana satu slot disisakan atau logika perhitungan melingkar digunakan untuk membedakan kondisi penuh dan kosong pada antrean berbasis pointer.

Operasi Penambahan (Enqueue)

Fungsi `enqueue` bertugas memasukkan data baru ke dalam antrean. Algoritma pertama-tama memastikan antrean tidak penuh melalui pengecekan `isFull`; jika penuh, program melempar `std::overflow_error`. Jika antrean kosong, baik `front` maupun `rear` diarahkan ke alamat awal array. Jika sudah berisi, pointer `rear` dipindahkan ke posisi berikutnya secara melingkar menggunakan rumus `(q->rear - q->data + 1) % MAX` sebelum nilai disimpan. Hal ini memastikan elemen baru selalu menempati posisi paling belakang dalam urutan antrean.

Operasi Penghapusan dan Akses (Dequeue, Front, Back)

Operasi `dequeue` dilakukan untuk mengeluarkan elemen terdepan. Jika antrean kosong, program melempar `std::underflow_error`. Jika hanya terdapat satu elemen tersisa (ketika `front == rear`), kedua pointer dikembalikan ke `nullptr` melalui fungsi init. Jika tidak, pointer front digeser ke posisi berikutnya secara melingkar. Justifikasi dari alur ini adalah bahwa perpindahan pointer `front` secara efektif menghapus akses ke data lama tanpa harus menghapus nilai fisik di memori. Sementara itu, fungsi `front` dan `back` digunakan untuk mengambil nilai pada posisi yang ditunjuk masing-masing pointer tanpa mengubah struktur antrean.

3. Kompleksitas dan Notasi O(1)

Seluruh fungsi dalam `queue.cpp` memiliki kompleksitas Waktu O(1) dan Ruang O(1). Kecepatan eksekusi bersifat konstan karena setiap operasi (seperti `enqueue` dan `dequeue`) langsung mengakses alamat memori melalui pointer tanpa melalui proses perulangan. Penggunaan memori juga tetap stabil karena array yang dipesan bersifat statis dan ukurannya tidak berubah selama program dijalankan.

SOAL 4 : Si Palui dan Deretan Angka

Time Limit: 1.0 s
Memory Limit: 64 MB

Si Palui memiliki sebuah deretan angka yang tersusun rapi. Ia tertarik untuk mengamati sebagian angka secara berurutan dengan jumlah tertentu.

Ia memilih sejumlah k angka yang bersebelahan, lalu menghitung jumlahnya. Setelah itu, ia menggeser pilihannya satu langkah ke kanan dan kembali menghitung jumlah dari angka-angka yang terpilih.

Proses ini dilakukan terus hingga tidak memungkinkan lagi untuk memilih k angka berurutan.

Sebagai asistennya, bantulah Si Palui untuk menentukan hasil perhitungan tersebut.

Batasan

- $1 \leq n \leq 100$
- $-1 \leq k \leq n$
- $-1000 \leq Ai \leq 1000$

Keterangan:

- n: jumlah elemen
- k: ukuran jendela
- Ai: elemen array

Format Input

n k

A1 A2 ... An

Keterangan:

- Baris pertama n menyatakan jumlah atau panjang array. Lalu k besar atau jumlah elemen
- dalam satu kelompok yang dibentuk
- Baris kedua berisi simbol A sebanyak n buah

Format Output

Cetak hasil tiap masing-masing kelompok dalam satu baris dipisahkan dengan spasi

Contoh Kasus

Input	Output
5 3 1 2 3 4 5	6 9 12
5 5 1 2 3 4 5	15
6 2 4 -1 2 -3 5 1	3 1 -1 2 6

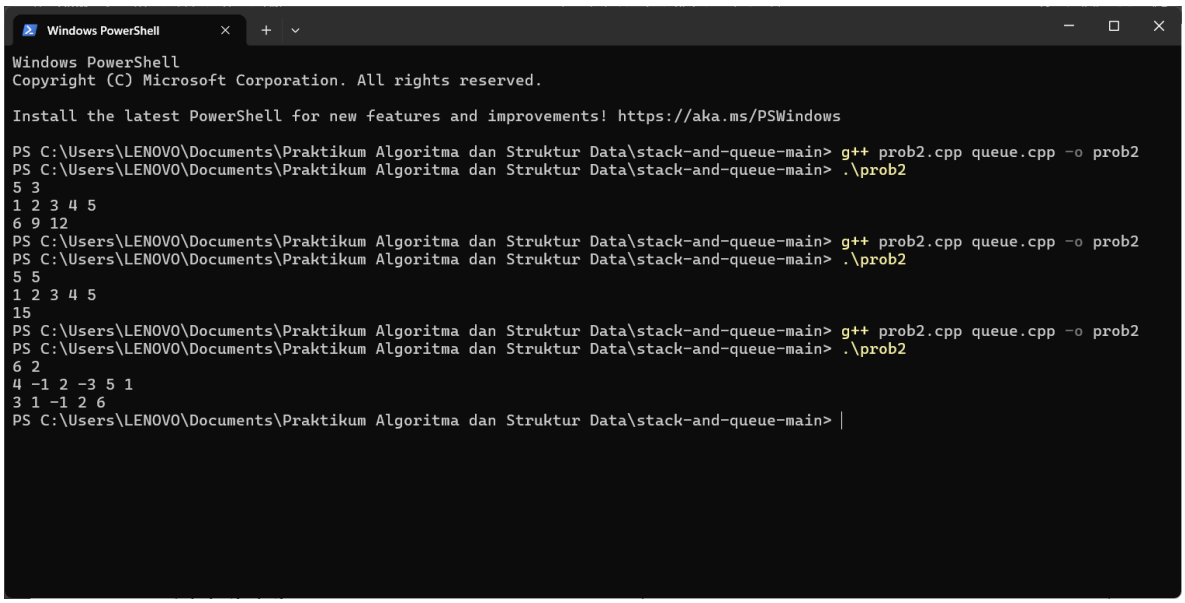
A. Source Code

Tabel 4. Source Code prob2.cpp

```
#include <iostream>
#include "queue.h"

using namespace std;

int main() {
    int n, k;
    if (!(cin >> n >> k)) return 0;
    if (k < 1) return 0;

    Queue q;
    init(&q);

    int sum = 0;

    for (int i = 0; i < n; i++) {
        int x;
        cin >> x;

        enqueue(&q, x);
        sum += x;

        if (i >= k) {
            sum -= front(&q);
            dequeue(&q);
        }

        if (i >= k - 1) {
            if (i > k - 1) cout << " ";
            cout << sum;
```

```
31            }
32        }
33        cout << endl;
34
35        return 0;
36    }
```

B. Output Program



Gambar 2. Screenshot Output Soal 4

C. Pembahasan

1. Penjelasan Baris Kode

Baris [1-2]: Digunakan untuk menyertakan pustaka `iostream` untuk keperluan input-output serta file header `queue.h` agar program dapat menggunakan fungsi antrean yang telah diimplementasikan sebelumnya.

Baris [4-6]: Perintah `using namespace std` digunakan untuk mempermudah penulisan fungsi standar, dan baris 6 merupakan titik awal dimulainya eksekusi program (`main`).

Baris [7-9]: Program membaca variabel `n` (jumlah total angka) dan `k` (ukuran jendela/kelompok). Jika input tidak valid atau nilai `k` kurang dari `1`, program akan langsung berhenti untuk menghindari kesalahan perhitungan.

Baris [11-14]: Deklarasi variabel `q` bertipe `Queue` diikuti dengan pemanggilan fungsi `init` untuk menyiapkan antrean dalam kondisi kosong. Variabel `sum` disiapkan dengan nilai awal 0 untuk menghitung total angka di dalam jendela `k` secara dinamis.

Baris [16-18]: Memulai perulangan sebanyak `n` kali. Di setiap putaran, program membaca satu angka dari input dan menyimpannya ke dalam variabel `x`.

Baris [20-21]: Angka `x` dimasukkan ke dalam antrean menggunakan fungsi `enqueue` dan nilainya ditambahkan ke variabel `sum`. Pada tahap ini, `sum` menyimpan total angka yang baru saja masuk.

Baris [23-26]: Bagian ini merupakan inti dari logika Sliding Window. Jika jumlah angka yang diproses sudah melebihi ukuran jendela `k`, program akan mengambil angka paling depan dari antrean menggunakan `front`, menguranginya dari variabel `sum`, lalu menghapusnya dengan `dequeue`. Hal ini memastikan bahwa `sum` selalu hanya berisi total dari `k` angka terakhir.

Baris [28-31]: Jika angka yang diproses sudah mencapai minimal `k` elemen, program akan mencetak nilai `sum` saat itu. Kondisi `if (i > k - 1)` digunakan untuk memastikan spasi hanya dicetak di antara angka, bukan di awal atau di akhir baris.

Baris [33-36]: Program mencetak baris baru setelah semua perhitungan selesai, kemudian mengakhiri eksekusi dengan mengembalikan nilai 0.

2. Analisis Kompleksitas

Kompleksitas Waktu O(n) :
Program ini berjalan dalam waktu linear karena hanya menggunakan satu perulangan tunggal yang memproses n elemen input. Di dalam perulangan tersebut, seluruh operasi antrean (`enqueue`, `dequeue`, `front`) dijalankan secara instan dalam waktu konstan atau O(1), sehingga total waktu eksekusi hanya bergantung pada banyaknya jumlah angka yang dimasukkan.

Kompleksitas Ruang O(1):
Meskipun menggunakan struktur data antrean, kapasitas memori yang digunakan bersifat statis dan sudah ditentukan sejak awal melalui konstanta `MAX` pada file *header*. Karena jumlah pemakaian memori ini tetap dan tidak bertambah secara dinamis mengikuti besar kecilnya input dari pengguna, maka kompleksitas ruangnya dikategorikan sebagai O(1) atau waktu konstan.

TAUTAN GITHUB

https://github.com/DSA26-ULM/module-2-stack-and-queue-nullrym